

Tutorial 1

DSC 140B, Spring 2025

TA: Abhilash, Divyansh
4/9/2025 @ Center Hall 216

Outline

- Environment setup:
 - Local setup with Conda
 - Google Colab
- Introduction to Numpy
- Introduction to Latex

Environment setup - locally

TA: Abhilash



Outline

- Installing conda
- Creating a conda environment
- Creating Jupyter Notebooks
- Setting up conda environment for Jupyter Notebooks

Environment setup: Local setup with Conda

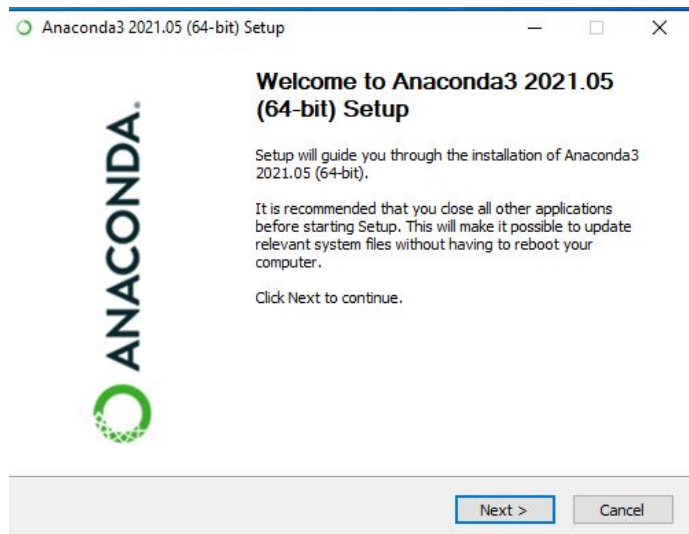
 Windows	 Mac	 Linux
Python 3.12	Python 3.12	Python 3.12
↓ 64-Bit Graphical Installer (912.3M)	↓ 64-Bit (Apple silicon) Graphical Installer (704.7M) ↓ 64-Bit (Apple silicon) Command Line Installer (707.3M) ↓ 64-Bit (Intel chip) Graphical Installer (734.7M) ↓ 64-Bit (Intel chip) Command Line Installer (731.2M)	↓ 64-Bit (x86) Installer (1007.9M) ↓ 64-Bit (AWS Graviton2 / ARM64) Installer (800.6M) ↓ 64-bit (Linux on IBM Z & LinuxONE) Installer (425.8M)

Visit <https://www.anaconda.com/download> and download the appropriate installer for your operating system (Windows, macOS, or Linux)

Environment setup: Local setup with Conda (Windows)

Run the installer and follow the prompts

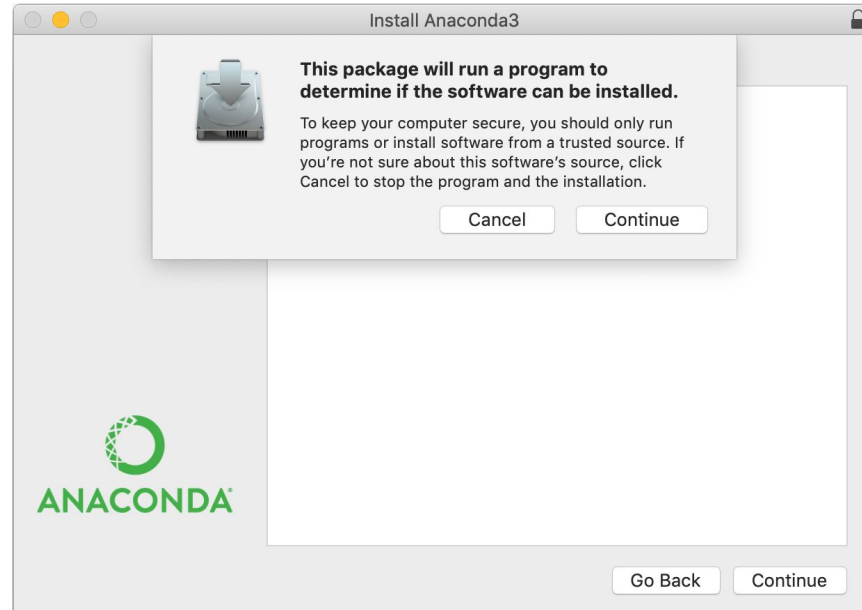
- Double-click the `.exe` file



Environment setup: Local setup with Conda (macOS)

Run the installer and follow the prompts

- For command line installer: Run `bash Anaconda3-2024.10-1-MacOSX-arm64.sh` in Terminal
- For Graphical installer: Double click on the *.pkg file



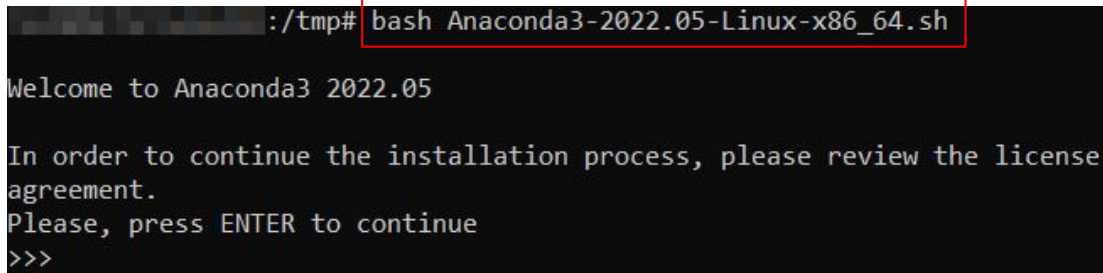
Environment setup: Local setup with Conda (Linux)

Run `bash Anaconda3-2024.10-1-Linux-x86_64.sh` in Terminal

Choose initialization option **YES**

- Conda modifies your shell configuration to initialize conda whenever you open a new shell and to recognize conda commands automatically.

Close and re-open your terminal window for the installation to fully take effect.

A terminal window with a black background and white text. The prompt is `:/tmp#`. The command `bash Anaconda3-2022.05-Linux-x86_64.sh` is entered and highlighted with a red rectangular box. The output shows the Anaconda3 2022.05 welcome message and a license agreement screen.

```
:/tmp# bash Anaconda3-2022.05-Linux-x86_64.sh

Welcome to Anaconda3 2022.05

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
>>>
```

Environment setup: Local setup with Conda

Verify Conda Installation

- Open a new terminal window (command prompt for Windows) and type `conda --version`
- This should display the installed Conda version

```
Abhilash@DESKTOP-LS7ILOD MINGW64 ~  
$ conda --version  
conda 4.10.3  
  
Abhilash@DESKTOP-LS7ILOD MINGW64 ~  
$ conda info  
  
active environment : None  
shell level : 0  
user config file : C:\Users\Abhilash\.condarc  
populated config files :  
conda version : 4.10.3  
conda-build version : 3.20.5  
python version : 3.8.5.final.0  
virtual packages : __cuda=12.8=0  
                  __win=0=0  
                  __archspec=1=x86_64  
base environment : C:\Users\Abhilash\anaconda3 (writable)  
conda av data dir : C:\Users\Abhilash\anaconda3\etc\conda  
conda av metadata url : None  
channel URLs : https://repo.anaconda.com/pkgs/main/win-64  
              https://repo.anaconda.com/pkgs/main/noarch  
              https://repo.anaconda.com/pkgs/r/win-64
```

Environment setup: Creating Conda Environment

Create a New Environment

Run `conda create --name myenv python=3.10`

Replace `myenv` with your preferred environment name and `3.10` with your desired Python version.

```
Abhilash@DESKTOP-LS7TLOD-MINGW64 ~
$ conda create --name dsc140b python=3.10
Collecting package metadata (current_repodata.json): ...working... done
Solving environment: ...working... done

## Package Plan ##

  environment location: C:\Users\Abhilash\anaconda3\envs\dsc140b

  added / updated specs:
    - python=3.10

The following NEW packages will be INSTALLED:

  bzip2                pkgs/main/win-64::bzip2-1.0.8-h2bbff1b_6
  ca-certificates      pkgs/main/win-64::ca-certificates-2025.2.25-haa95532_0
  libffi               pkgs/main/win-64::libffi-3.4.4-hd77b12b_1
  openssl              pkgs/main/win-64::openssl-3.0.16-h3f729d1_0
  pip                  pkgs/main/win-64::pip-25.0-py310haa95532_0
  python               pkgs/main/win-64::python-3.10.16-h4607a30_1
  setuptools           pkgs/main/win-64::setuptools-75.8.0-py310haa95532_0
  sqlite               pkgs/main/win-64::sqlite-3.45.3-h2bbff1b_0
  tk                   pkgs/main/win-64::tk-8.6.14-h0416ee5_0
  tzdata               pkgs/main/noarch::tzdata-2025a-h04d1e81_0
  vc                   pkgs/main/win-64::vc-14.42-haa95532_4
  vs2015_runtime       pkgs/main/win-64::vs2015_runtime-14.42.34433-he0abc0d_4
  wheel                pkgs/main/win-64::wheel-0.45.1-py310haa95532_0
  xz                   pkgs/main/win-64::xz-5.6.4-h4754444_1
  zlib                 pkgs/main/win-64::zlib-1.2.13-h8cc25b3_1
```

```
Proceed ([y]/n)? y
```

Environment setup: Creating Conda Environment

Activate the newly created environment: run `conda activate myenv`

Install the packages you need: run `conda install numpy pandas matplotlib`

To deactivate the environment: `conda deactivate`

```
Abhilash@DESKTOP-LS7ILOD MINGW64 ~  
$ conda activate dsc140b  
(dsc140b)  
Abhilash@DESKTOP-LS7ILOD MINGW64 ~  
$ conda install numpy pandas matplotlib  
Collecting package metadata (current_repodata.json): ...working... done  
Solving environment: ...working... done  
  
## Package Plan ##  
  
environment location: C:\Users\Abhilash\anaconda3\envs\dsc140b  
  
added / updated specs:  
- matplotlib  
- numpy  
- pandas
```

Environment setup: Conda Environment

Managing Your Environment

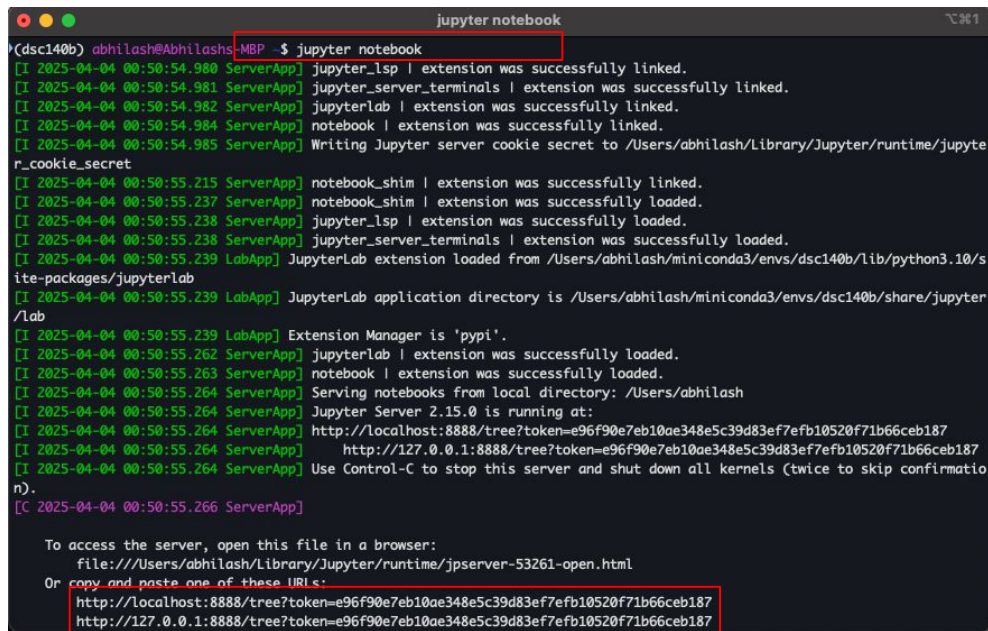
Some useful commands for managing your environment:

- List all environments: `conda env list`
- List installed packages: `conda list`
- Export environment: `conda env export > environment.yml`
- Create environment from file: `conda env create -f environment.yml`
- To clean unused packages and caches: `conda clean -a`

That's it! You now have a working Conda setup for managing Python environments.

Installing Jupyter Notebooks

- By default, jupyter is installed when you installed conda.
- If not, install jupyter by running
 - `pip install jupyterlab notebook`



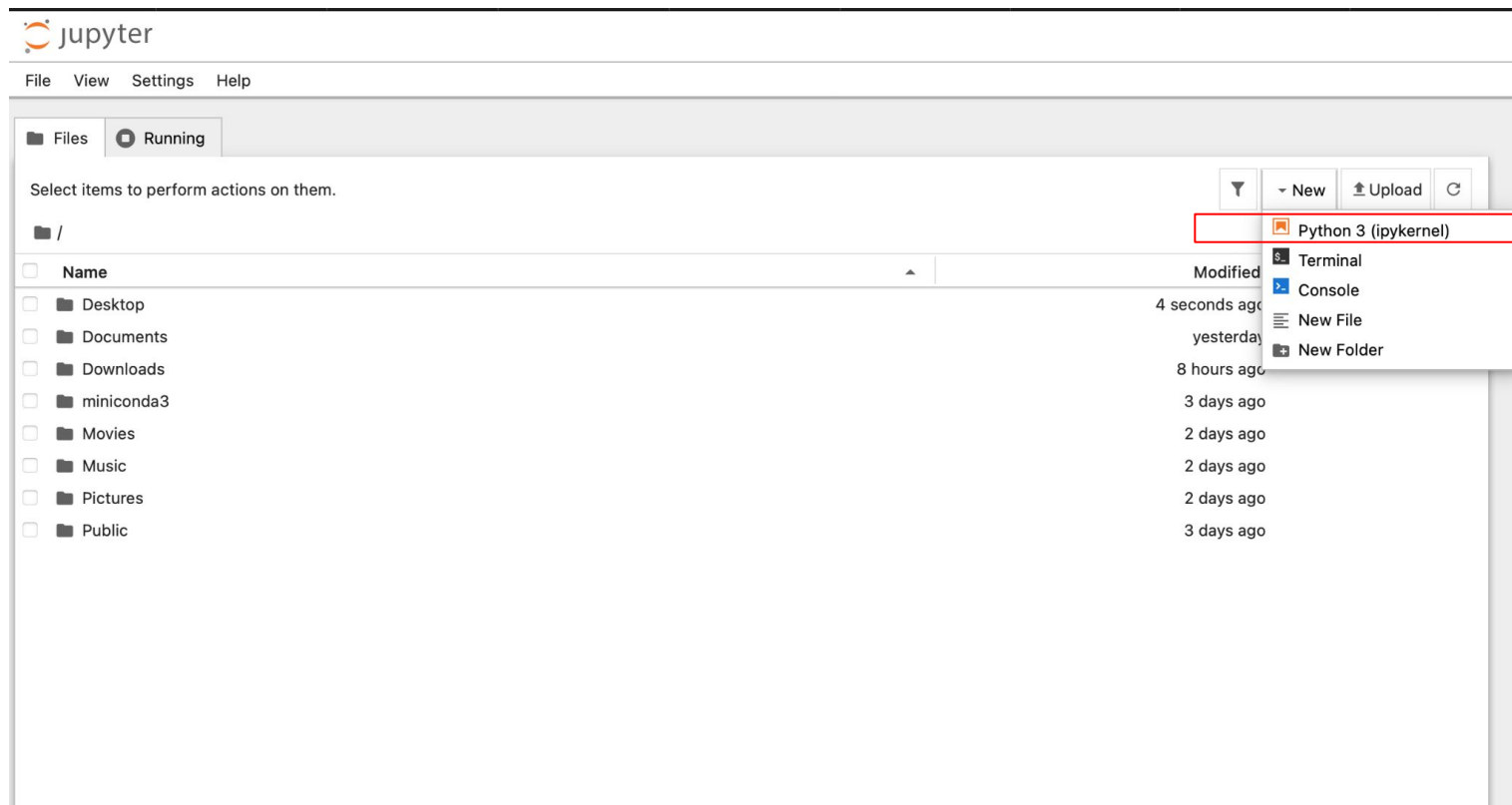
```

jupyter notebook
(dsc140b) abhilash@Abhilashs-MBP ~$ jupyter notebook
[I 2025-04-04 00:50:54.980 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2025-04-04 00:50:54.981 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2025-04-04 00:50:54.982 ServerApp] jupyterlab | extension was successfully linked.
[I 2025-04-04 00:50:54.984 ServerApp] notebook | extension was successfully linked.
[I 2025-04-04 00:50:54.985 ServerApp] Writing Jupyter server cookie secret to /Users/abhilash/Library/Jupyter/runtime/jupyter_cookie_secret
[I 2025-04-04 00:50:55.215 ServerApp] notebook_shim | extension was successfully linked.
[I 2025-04-04 00:50:55.237 ServerApp] notebook_shim | extension was successfully loaded.
[I 2025-04-04 00:50:55.238 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2025-04-04 00:50:55.238 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2025-04-04 00:50:55.239 LabApp] JupyterLab extension loaded from /Users/abhilash/miniconda3/envs/dsc140b/lib/python3.10/site-packages/jupyterlab
[I 2025-04-04 00:50:55.239 LabApp] JupyterLab application directory is /Users/abhilash/miniconda3/envs/dsc140b/share/jupyter/lab
[I 2025-04-04 00:50:55.239 LabApp] Extension Manager is 'pypi'.
[I 2025-04-04 00:50:55.262 ServerApp] jupyterlab | extension was successfully loaded.
[I 2025-04-04 00:50:55.263 ServerApp] notebook | extension was successfully loaded.
[I 2025-04-04 00:50:55.264 ServerApp] Serving notebooks from local directory: /Users/abhilash
[I 2025-04-04 00:50:55.264 ServerApp] Jupyter Server 2.15.0 is running at:
[I 2025-04-04 00:50:55.264 ServerApp] http://localhost:8888/tree?token=e96f90e7eb10ae348e5c39d83ef7efb10520f71b66ceb187
[I 2025-04-04 00:50:55.264 ServerApp] http://127.0.0.1:8888/tree?token=e96f90e7eb10ae348e5c39d83ef7efb10520f71b66ceb187
[I 2025-04-04 00:50:55.264 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).

[C 2025-04-04 00:50:55.266 ServerApp]

To access the server, open this file in a browser:
file:///Users/abhilash/Library/Jupyter/runtime/jpserver-53261-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=e96f90e7eb10ae348e5c39d83ef7efb10520f71b66ceb187
http://127.0.0.1:8888/tree?token=e96f90e7eb10ae348e5c39d83ef7efb10520f71b66ceb187
  
```

Creating Jupyter Notebooks



Using Conda environment in jupyter notebook

- Activate the environment
- Run
 - `conda install -c anaconda ipykernel`
 - `python -m ipykernel install --user --name=firstEnv`
- Replace `firstEnv` with your environment name



Quit

Logout

Files

Running

Clusters

Select items to perform actions on them.

0 /

	Name
☐	4.2_NoopurRK_DOS
☐	anaconda3
☐	Applications
☐	bitcoinsimulator
☐	Desktop
☐	Documents

Upload New

Notebook:

- Python 3
- dnn2
- firstEnv**
- trial

Other:

- Text File
- Folder
- Terminal

Environment setup - Colab

TA: Divyansh



Outline

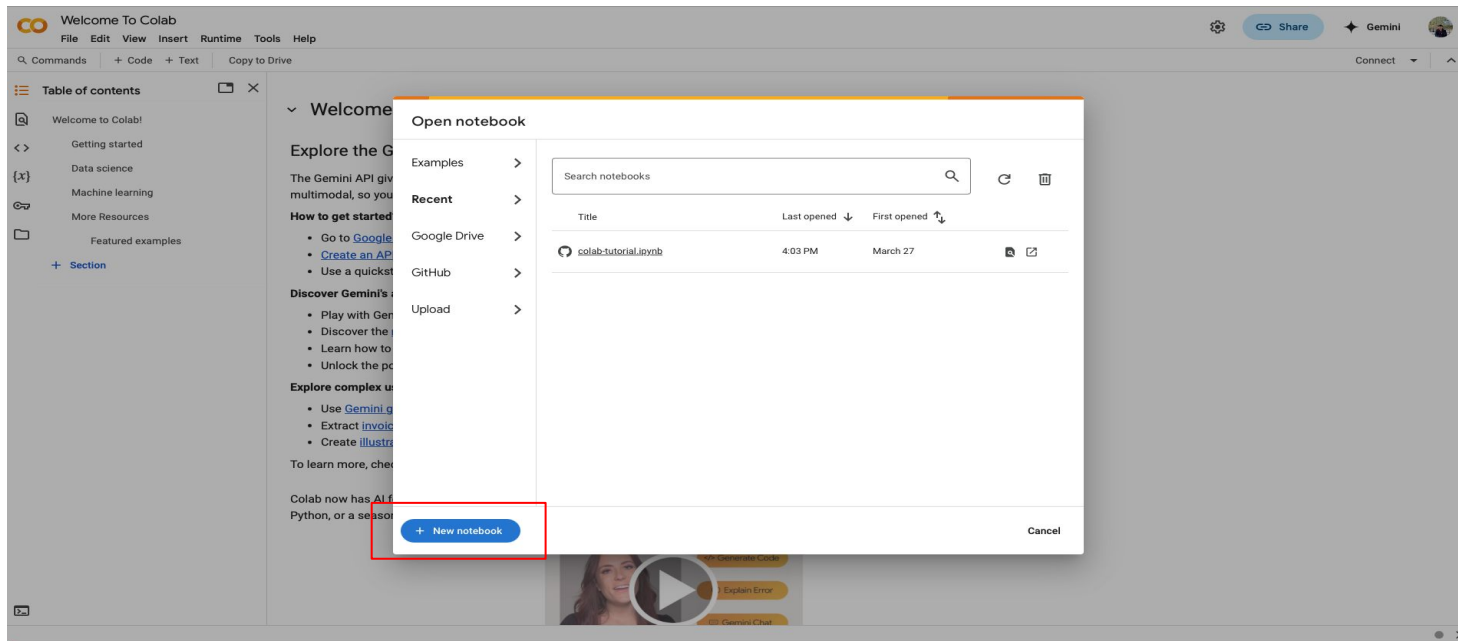
- Google Colab Introduction
- Getting Started
- Running python code
- Install Packages
- Adding Files
- Changing Runtimes
- Markdown

Google Colab: Introduction

- Colab is a cloud-based python notebook environment
 - Free for general-use
 - Minimal setup
 - Free limited-access to TPUs to accelerate ML training
- Limitations
 - Notebooks can run for a maximum of 12 hours
 - Lower allowed runtime if using GPUs

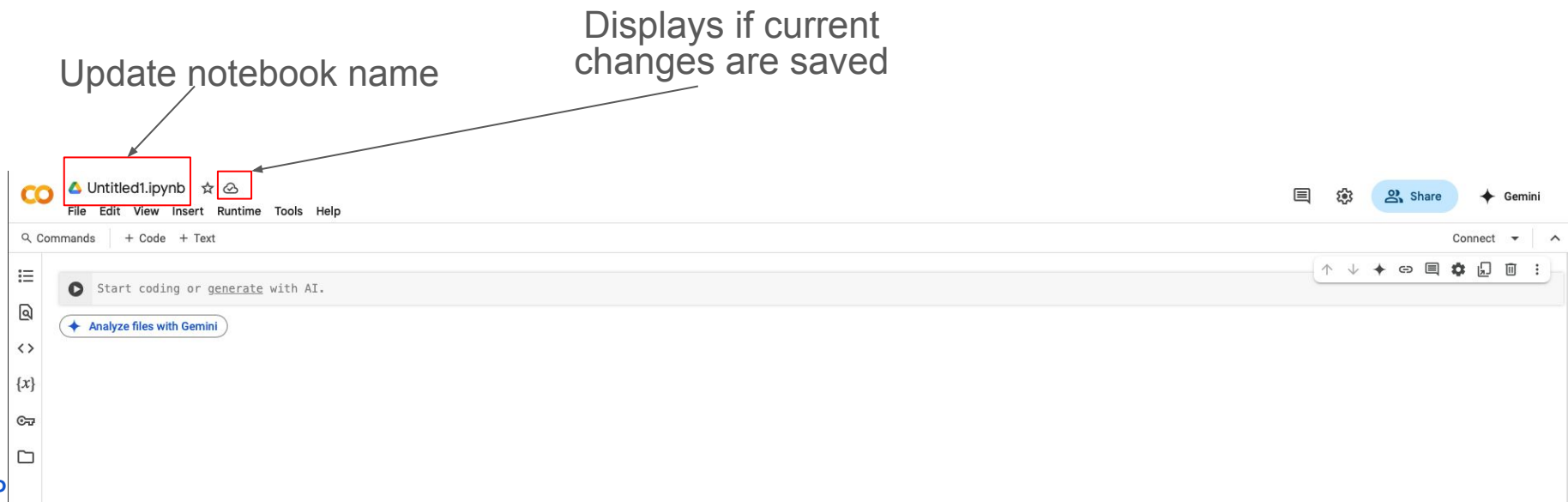
Google Colab: Getting started

- Go to colab.google.com and click on “New notebook”



Google Colab: Getting started

- Update name of notebook
- Save changes with **cmd+s** (mac) or **ctrl+s** (windows)



Google Colab: Running python code



Step 2: Run cell

Google Colab: Install Packages

- Install packages by running command: **!pip install <package-name>**
- Install multiple packages: **!pip install <pn1> <pn2> ...**

```
!pip install torch
```

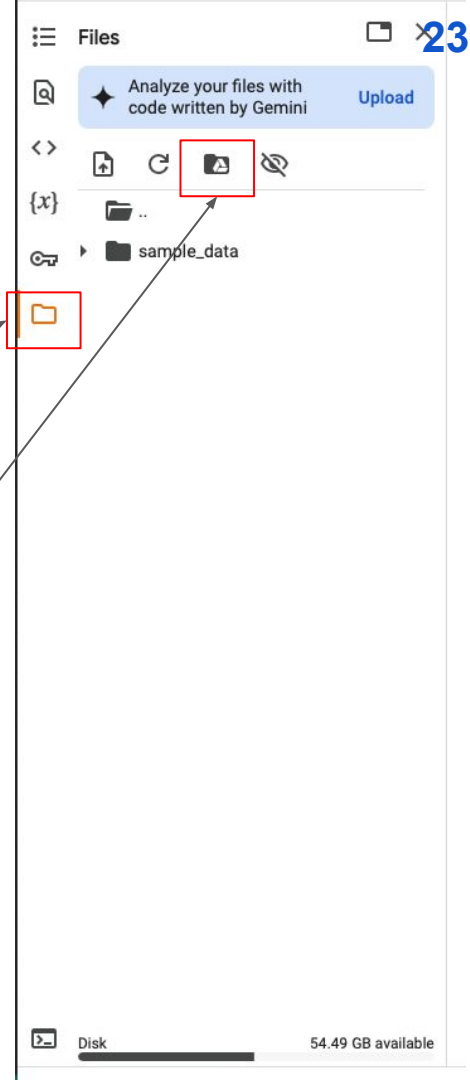
```
... Requirement already satisfied: torch in /usr/local/lib/python3.11/dist-packages (2.6.0+cu124)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch) (4.13.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch) (2025.3.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)
```

Google Colab: Adding files

- Connect to google drive
 - WARNING: The files are deleted if stored in temporary environment

Step 1: Click on files icon

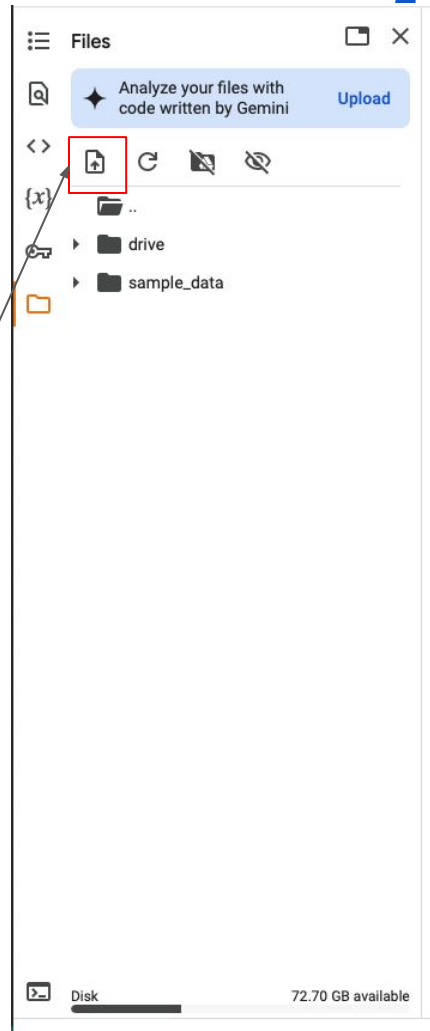
Step 2: Click on google drive icon



Google Colab: Adding files

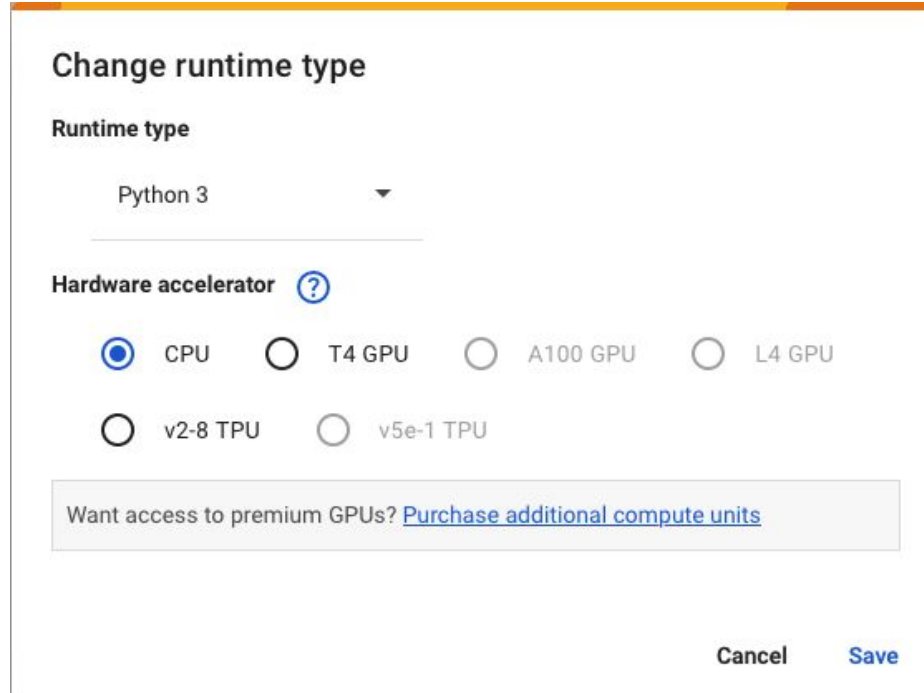
- Google drive will be mounted as “drive” folder
 - Files added into drive folder can be accessed in different notebooks and across sessions
- You can upload files from your local computer using the upload button

Step 3: Upload files



Google Colab: Changing runtimes

- Click on **Runtime > Change runtime type**



The screenshot shows the 'Change runtime type' dialog box in Google Colab. It has a title bar with an orange gradient. Inside, the title 'Change runtime type' is at the top. Below it is the 'Runtime type' section with a dropdown menu currently set to 'Python 3'. The 'Hardware accelerator' section has a help icon (?) and five radio button options: 'CPU' (selected), 'T4 GPU', 'A100 GPU', 'L4 GPU', and 'v2-8 TPU'. Below these is 'v5e-1 TPU'. At the bottom of the dialog are 'Cancel' and 'Save' buttons. A light gray box at the bottom contains the text 'Want access to premium GPUs?' followed by a blue link 'Purchase additional compute units'.

Change runtime type

Runtime type

Python 3

Hardware accelerator ?

☒ CPU ☐ T4 GPU ☐ A100 GPU ☐ L4 GPU

☐ v2-8 TPU ☐ v5e-1 TPU

Want access to premium GPUs? [Purchase additional compute units](#)

Cancel Save

Google Colab: Markdown

- Colab has two types of cells: **text** and **code**.
- Text cells are formatted using a simple markup language called **Markdown**.



Google Colab: Markdown

Heading	# H1 ## H2 ### H3
Bold	**bold text**
Italic	<i>*italicized text*</i>
Blockquote	> blockquote
Ordered List	1. First item 2. Second item
Unordered List	- First item - Second item - Third item
Code	`code`
Horizontal Rule	---
Link	[title] (https://www.example.com)
Image	![alt text] (image.jpg)

H1

H2

H3

bold text

italicized text

blockquote

1. First item
2. Second item

- First item
- Second item
- Third item

code

title



Reference:
<https://www.markdownguide.org/cheat-sheet/>

DEMO

Intro to Numpy

TA: Abhilash



Outline

- Creating Arrays
- Array Indexing
- Basic Math
- Linear Algebra
- Eigenvalues and eigenvectors
- Singular Value Decomposition

Numpy

- NumPy's main object is the homogeneous multidimensional array.
- It is a table of elements (usually numbers), all of the same type, indexed by a tuple of non-negative integers.
- In NumPy dimensions are called axes.

```
import numpy as np

a = np.array([1, 2, 3]) # Create a rank 1 array
print(type(a))         # Prints "<class 'numpy.ndarray'>"
print(a.shape)         # Prints "(3,)"
print(a[0], a[1], a[2]) # Prints "1 2 3"
a[0] = 5               # Change an element of the array
print(a)               # Prints "[5, 2, 3]"

b = np.array([[1,2,3],[4,5,6]]) # Create a rank 2 array
print(b.shape)                # Prints "(2, 3)"
print(b[0, 0], b[0, 1], b[1, 0]) # Prints "1 2 4"
```

Numpy: Vectors

- Creating a one-dimensional array

```
import numpy as np

# Create a row vector
vector_row = np.array([1, 2, 3])
# vector_row = [1 2 3]
# vector_row.shape = (3,)
```

```
# Create a column vector
vector_column = np.array([[1], [2], [3]])
# vector_column = [[1]
#                  [2]
#                  [3]]
# vector_column.shape = (3, 1)
```

Numpy: Arrays

Numpy also provides many functions to create arrays:

```
import numpy as np

a = np.zeros((2,2))    # Create an array of all zeros
print(a)              # Prints "[[ 0.  0.]
                      #           [ 0.  0.]]"

b = np.ones((1,2))    # Create an array of all ones
print(b)              # Prints "[[ 1.  1.]]"

c = np.full((2,2), 7) # Create a constant array
print(c)              # Prints "[[ 7.  7.]
                      #           [ 7.  7.]]"

d = np.eye(2)         # Create a 2x2 identity matrix
print(d)              # Prints "[[ 1.  0.]
                      #           [ 0.  1.]]"

e = np.random.random((2,2)) # Create an array filled with random values
print(e)              # Might print "[[ 0.91940167  0.08143941]
                      #           [ 0.68744134  0.87236687]]"
```

Numpy: Array Indexing

```
import numpy as np

# Create the following rank 2 array with shape (3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]
a = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12]])

# Use slicing to pull out the subarray consisting of the first 2 rows
# and columns 1 and 2; b is the following array of shape (2, 2):
# [[2 3]
#  [6 7]]
b = a[:2, 1:3]

# A slice of an array is a view into the same data, so modifying it
# will modify the original array.
print(a[0, 1]) # Prints "2"
b[0, 0] = 77   # b[0, 0] is the same piece of data as a[0, 1]
print(a[0, 1]) # Prints "77"
```

Numpy uses zero indexing and arrays may be multidimensional, you must specify a slice for each dimension of the array.

Numpy: Basic math

- Basic mathematical functions operate elementwise on arrays, and are available both as operator overloads and as functions in the numpy module

```
import numpy as np

x = np.array([[1,2],[3,4]], dtype=np.float64)
y = np.array([[5,6],[7,8]], dtype=np.float64)

# Elementwise sum; both produce the array
# [[ 6.0  8.0]
#  [10.0 12.0]]
print(x + y)
print(np.add(x, y))

# Elementwise difference; both produce the array
# [[-4.0 -4.0]
#  [-4.0 -4.0]]
print(x - y)
print(np.subtract(x, y))

# Elementwise product; both produce the array
# [[ 5.0 12.0]
#  [21.0 32.0]]
print(x * y)
print(np.multiply(x, y))

# Elementwise division; both produce the array
# [[ 0.2          0.33333333]
#  [ 0.42857143  0.5        ]]
print(x / y)
print(np.divide(x, y))

# Elementwise square root; produces the array
# [[ 1.          1.41421356]
#  [ 1.73205081  2.        ]]
print(np.sqrt(x))
```

Numpy: Basic math

- We instead use the **dot** function to compute inner products of vectors, to multiply a vector by a matrix, and to multiply matrices.
- **dot** is available both as a function in the numpy module and as an instance method of array objects.

```
import numpy as np

x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

v = np.array([9,10])
w = np.array([11, 12])

# Inner product of vectors; both produce 219
print(v.dot(w))
print(np.dot(v, w))

# Matrix / vector product; both produce the rank 1 array [29 67]
print(x.dot(v))
print(np.dot(x, v))

# Matrix / matrix product; both produce the rank 2 array
# [[19 22]
# [43 50]]
print(x.dot(y))
print(np.dot(x, y))
```

Numpy: Basic math

- Note that numpy defaults to performing many operators in an "*element wise*" fashion.
- For example, in math, the notation A^2 denotes the matrix $A \cdot A$, that is, the product of A with itself.
- In Python, we can raise something to a power using the `**` operator. But if we write `A**2`, we do not get A^2

```
1 import numpy as np
2
3 a = np.array([[1, 2], [3, 4]])
4 print(a*a)
5 # [[ 1  4]
6 #   [ 9 16]]
7 print(a**2)
8 # [[ 1  4]
9 #   [ 9 16]]
10 print(a @ a)
11 # [[ 7 10]
12 #   [15 22]]
```

Another way to
perform **dot**
operation

Numpy: Linear Algebra

```
import numpy as np

vector = np.array([1, 2, 3])
print(vector) # prints [1, 2, 3]

vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])
print(np.dot(vector1, vector2)) # prints 32

# Cross Product
vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])
print(np.cross(vector1, vector2)) # prints [-3, 6, -3]

# Norm: The norm of a vector is a scalar that represents the "length" of the
# vector. In NumPy, you can compute the norm using the numpy.linalg.norm function,
vector = np.array([1, 2, 3])
print(np.linalg.norm(vector)) # prints 3.74165738677

# The inner product of two vectors is a matrix that is computed by multiplying
# the first vector by the transpose of the second vector. In NumPy, you can
# compute the inner product using the numpy.inner function
vector1 = np.array([1, 2, 3])
vector2 = np.array([4, 5, 6])
print(np.inner(vector1, vector2)) # prints 32
```

Numpy: Linear Algebra

```
import numpy as np
# Scalar Multiplication
matrix = np.array([[1, 2], [3, 4]])
print(2 * matrix) # prints [[2, 4], [6, 8]]

# Matrix Multiplication
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])
print(np.dot(matrix1, matrix2)) # prints [[19, 22], [43, 50]]

# Transpose
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(np.transpose(matrix)) # prints [[1, 4], [2, 5], [3, 6]]

# inverse of a matrix
a = np.array([[1, 2], [3, 4]])
a_inv = np.linalg.inv(a)
print(a_inv)
# prints [[-2., 1. ], [ 1.5, -0.5]]
```

NumPy takes care of all the messy details behind the scenes.

Numpy: Eigenvalues and eigenvectors

- In NumPy, you can easily compute eigenvalues and eigenvectors using the *numpy.linalg.eig* function

```
import numpy as np

A = np.array([[4, 2], [1, 3]])
eigenvalues, eigenvectors = np.linalg.eig(A)
# eigenvalues [5., 2.]
# eigenvectors [[ 0.89442719, -0.70710678], [ 0.4472136,  0.70710678]]
```

- eigenvectors[:,i] is the eigenvector corresponding to the eigenvalue eigenvalues[i]

Numpy: Singular Value Decomposition

- U will be a matrix containing the left singular vectors, S will be a 1D array containing the singular values, and Vt will be the transpose of a matrix containing the right singular vectors.

```
import numpy as np

A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

U, S, Vt = np.linalg.svd(A)

# U = [[-0.21483724  0.88723069  0.40824829]
#       [-0.52058739  0.24964395 -0.81649658]
#       [-0.82633754 -0.38794278  0.40824829]]

# S = [1.68481034e+01 1.06836951e+00 4.41842475e-16]

# Vt = [[-0.47967118 -0.57236779 -0.66506441]
#        [-0.77669099 -0.07568647  0.62531805]
#        [-0.40824829  0.81649658 -0.40824829]]

sigma = np.diag(S)
reconstructed_A = np.dot(U, np.dot(sigma, Vt))
# reconstructed_A = [[1. 2. 3.]
#                    [4. 5. 6.]
#                    [7. 8. 9.]]
```

You can also reconstruct the original matrix using the computed matrices

Intro to Latex and OverLeaf

TA: Divyansh



Outline

- Introduction
- OverLeaf
- Syntax
- Commands
 - Sections
 - Text Formatting
 - Unordered List
 - Ordered List
 - Images
 - Math Mode
 - Subscript and Superscript

LaTeX: Introduction

- A markup language for typesetting documents.
- Widely used in academia, especially for:
 - Scientific papers
 - Theses
 - Technical reports
- Focuses on content, not appearance.
- Pronounced «Lah-tech» or «Lay-tech»

LaTeX: OverLeaf

- Cloud LaTeX management and compilation
- Great LaTeX editing tools!
- Free for basic use

Create new project

Overleaf
A Digital Science Solution

Features & Benefits ▾ Templates Plans & Pricing

You're on the free plan ⓘ Upgrade

All Projects

Search in all projects...

☐ Title	Owner	Last Modified ↓	Actions
--------------------------------	-------	-----------------	---------

LaTeX: OverLeaf

Step 1: Write your code here

Step 2: Code compilation

The screenshot displays the OverLeaf web interface. On the left, the 'Code Editor' tab is active, showing a LaTeX document with the following code:

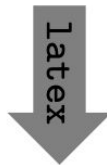
```
1 \documentclass{article}
2 \usepackage{graphicx} % Required for inserting images
3
4 \title{Demo}
5 \author{Divyansh Srivastava}
6 \date{April 2025}
7
8 \begin{document}
9
10 \maketitle
11
12 \section{Introduction}
13
14 \end{document}
15
```

On the right, the 'Recompile' button is highlighted with a red box. Below it, the preview window shows the rendered output of the LaTeX code, which includes the title 'Demo', the author 'Divyansh Srivastava', the date 'April 2025', and a section header '1 Introduction'.

LaTeX: Syntax

- You write your document in plain text with commands that describe its structure and meaning.
- The latex program processes your text and commands to produce a beautifully formatted document.

The rain in Spain falls `\emph{mainly}` on the plain.



The rain in Spain falls *mainly* on the plain.

LaTeX: Syntax

- Commands start with a backslash \

```
\begin{itemize}  
\item Tea  
\item Milk  
\item Biscuits  
\end{itemize}
```

- ▶ Tea
- ▶ Milk
- ▶ Biscuits

```
\begin{figure}  
\includegraphics{gerbil}  
\end{figure}
```



```
\begin{equation}  
\alpha + \beta + 1  
\end{equation}
```

$$\alpha + \beta + 1 \quad (1)$$

LaTeX: Sections

- Use section command to arrange your content
 - `\section{NAME}` starts a section
 - `\subsection{NAME}` starts a subsection
 - `\subsubsection{NAME}` starts a subsubsection

```
\section{Introduction}  
  
\section{DummySection1}  
  \subsection{SubSection1}  
  \subsection{SubSection2}  
  \subsection{SubSection3}  
    \subsubsection{SubSubSection1}  
  
\end{document}
```

```
1  Introduction  
2  DummySection1  
  2.1 SubSection1  
  2.2 SubSection2  
  2.3 SubSection3  
    2.3.1 SubSubSection1
```

LaTeX: Text Formatting

- Bold: `\textbf{...}`
- Italics: `\textit{...}`
- Underline: `\underline{...}`

```
Some of the \textbf{greatest}
discoveries in \underline{science}
were made by \textbf{\textit{accident}}.
```

➞ [Open this example in Overleaf.](#)

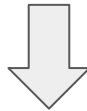
This example produces the following output:

Some of the **greatest** discoveries in science were made by *accident*.

LaTeX: Unordered List

- Unordered List: `\begin{itemize} ..... \end{itemize}`
- Add each item with `\item`

```
\begin{itemize}
  \item The individual entries are indicated with a black dot, a so-called
  bullet.
  \item The text in the entries may be of any length.
```

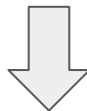


- The individual entries are indicated with a black dot, a so-called bullet.
- The text in the entries may be of any length.

LaTeX: Ordered List

- Ordered List: `\begin{enumerate} ..... \end{enumerate}`
- Add each item with `\item`

```
\begin{enumerate}  
  \item This is the first entry in our list.  
  \item The list numbers increase with each entry we add.  
\end{enumerate}
```



1. This is the first entry in our list.
2. The list numbers increase with each entry we add.

LaTeX: Images

- Requires the `graphicx` package, which provides the **`\includegraphics`** command.
- Supported graphics formats include JPEG, PNG and PDF



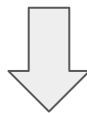
Not so Easy!

Figure 1: A happy face

LaTeX: Inline math mode

- Write within `$.....$` or `\begin{math}....\end{math}`

In physics, the mass-energy equivalence is stated
by the equation `$E=mc^2$`, discovered in 1905 by Albert Einstein.



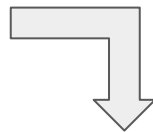
In physics, the mass-energy equivalence is stated by the equation $E = mc^2$, discovered in 1905 by Albert Einstein.

LaTeX: Display math mode

- Unnumbered: `\[.....\]` (add `$$....$$`)
- Numbered: `\begin{equation}....\end{equation}`

The mass-energy equivalence is described by the famous equation
`\[ E=mc^2 \]` discovered in 1905 by Albert Einstein.

In natural units ($c = 1$), the formula expresses the identity
`\begin{equation}`
`E=m`
`\end{equation}`



The mass-energy equivalence is described by the famous equation

$$E = mc^2$$

discovered in 1905 by Albert Einstein.

In natural units ($c = 1$), the formula expresses the identity

$$E = m$$

(1)

LaTeX: Display math mode

- Do not use $\\$\\$....\\$\\$$ for display math mode
- Old construct with limited math support
 - For example, QED at the end of proof doesn't work
 - Limited rendering support with **asmath** package

LaTeX: Superscript or Subscript

- `$text_{\text{subscript}}^{\text{superscript}}$`

`$a^{b}$, $a_{b}$, $a_{b}^{c}$`



a^b , a_b , a_b^c

DEMO